



JS



```
1 // JavaScript
3 const greeting = 'Hello, JavaScript!';
4
5 function greet(name) {
6   return `Hello, ${name}!`;
7 }
8
9 const message = greet('Developer');
10 console.log(message);
11
```



{ }

</>



OUR LADY OF FATIMA UNIVERSITY - ANTIPOLO
COLLEGE OF COMPUTER STUDIES



LESSON 1

INTRODUCTION TO JAVASCRIPT





JavaScript is a programming language used to make websites interactive and dynamic. It's one of the core technologies of the web, alongside HTML and CSS.

In simple terms:

- **HTML** builds the structure (like the skeleton of a webpage)
- **CSS** styles it (colors, layouts, fonts)
- **JavaScript** adds behavior (what happens when you click, scroll, type, etc.)

What JavaScript can do:

- Respond to user actions (clicks, typing, scrolling)
- Update content without reloading the page
- Create animations and games
- Communicate with servers (fetch data from APIs)
- Power web apps like maps, chats, and online editors





What is JavaScript?

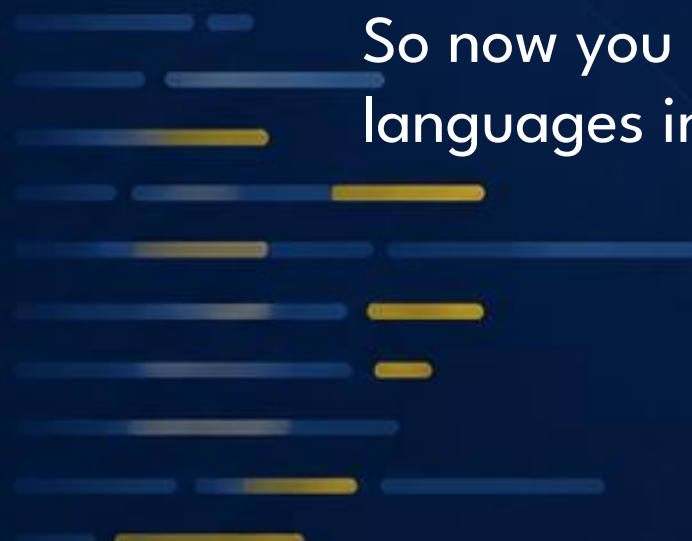
Let's finally define what JavaScript actually is:

- JavaScript is a lightweight, object-oriented programming language.
- It's a scripting language mainly used to make web pages interactive.
- It adds dynamic behavior — like animations, pop-ups, sliders, and form validation.
- Files end with the “.js” extension.
- JavaScript is also used in server-side development thanks to Node.js.

Where Does JavaScript Run?

Originally, JavaScript was designed to run inside web browsers — helping webpages react to user actions like clicks, scrolls, and inputs. But today, with tools like Node.js, it can also run outside the browser, powering web servers, desktop apps, and even IoT devices!

So now you know what JavaScript is, why we use it, and how it evolved. It's one of the most important languages in web development — and once you learn it, you'll be able to make websites truly come alive.



Introduction to JavaScript



JavaScript was created in **1995** by Brendan Eich while working at Netscape. He developed the first version in only **10 days** for the Netscape Navigator browser.

Early Names

JavaScript had several names before becoming official:

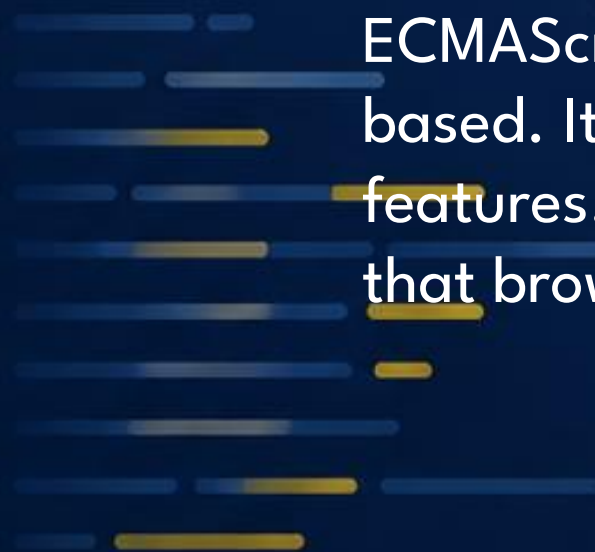
- **Mocha** – original project name
- **LiveScript** – temporary name
- **JavaScript** – final name, chosen partly for marketing because Java was popular at that time.

Standardization

In **1997**, JavaScript was standardized by ECMA International and became known officially as **ECMAScript**.

What is ECMAScript?

ECMAScript is a standard on which JavaScript is based! ECMAScript is the standard on which JavaScript is based. It was created so that everyone talking about JavaScript would refer to the same set of rules and features. You can think of ECMAScript as the official blueprint — and JavaScript as the working version that browsers actually run.





JavaScript syntax comprises a set of rules that define how to construct a JavaScript code. JavaScript can be implemented using JavaScript statements that are placed within the **<script>... </script>** HTML tags in a web page.

You can place the **<script>** tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the **<head>** tags.

The **<script>** tag alerts the browser program to start interpreting all the text between these tags as a script. A simple syntax of your JavaScript will appear as follows.

```
<html>
<head>
  <title> Your first JavaScript program </title>
</head>
<body>
  <script language = "javascript" type = "text/javascript">
    document.write("Hello World!")
  </script>
</body>
</html>
```





JavaScript Syntax



The JavaScript syntax defines two types of values:

1. Literals (Fixed Values)

Literals are values written directly into the program. They do not change unless you edit the code.

Used in Statements

`</> JavaScript`



```
document.write("welcome");  
let age = 21;
```

Here:

"Welcome" is a string literal

21 is a number literal



2. Variables (Variable Values)

Variables are containers used to store data values. Their values can change during program execution.

Examples

JavaScript



```
let name = "Arjay";  
let age = 21;
```

Changing Variable Values

JavaScript



```
let score = 50;  
score = 75;
```

The value of `score` changed from `50` to `75`.



JavaScript Syntax



JavaScript Variables

Variables are containers used to store data values in JavaScript. They allow you to save information and use it later in your program.

Rules:

- Must start with a letter, _, or \$
- Cannot start with a number
- No spaces
- Case-sensitive (name and Name are different)
- Hyphens are not allowed in JavaScript. They are reserved for subtractions.



JavaScript Statements



In JavaScript, **statements** are instructions that tell the computer what to do. A JavaScript program is made up of one or more statements.

Types of Statements

1. Variable Statement - Used to declare variables.
2. Assignment Statement - Used to assign a value.
3. Output Statement - Displays output.
4. Conditional Statement - Used to make decisions.
5. Loop Statement - Repeats code.

Statement Separator (;) - Statements are usually ended with a semicolon ;

Code Blocks ({ }) - Multiple statements can be grouped using { }





JavaScript comments are used to explain the purpose of the code. The **comments** are not executed as a part of program and these are solely meant for human developers to understand the code better.

You can use comments for various purposes, such as

- Explaining the purpose of a particular piece of code.
- Adding documentation to your code for yourself and others who may read it.
- Temporarily disabling or "commenting out" a block of code for testing or debugging without deleting it.

Types of Comments in JavaScript

- **Single-line comments** - We can start the single-line comment with the **double forward slash** (//). We can write comment between // and end of the line.
- **Multi-line comments** - The multi-line comment is useful when we require to comment the multiple lines of code or explain the larger codes. We can write multi-line comments between the /* and */.





JavaScript Variables



Variables are named containers used to store data values in JavaScript. They allow programs to save information, update it, and use it later.

Why Variables Are Important

- Variables can store:
- Numbers
- Text (strings)
- True/False values
- Arrays
- Objects

Declaring Variables

JavaScript provides three ways to declare variables:

1. Let - Used when the value may change.
2. Const - Used when the value should not be reassigned.
3. Var - Older way to declare variables.



JavaScript Datatypes



In JavaScript, **data types** define the kind of value a variable can hold, such as numbers, text, or true/false values.

Main JavaScript Data Types

Primitive Data Types

1. Number - Used for integers and decimals.
2. String - Used for text.
3. Boolean - Represents true or false.
4. Undefined - A variable declared but no value assigned.
5. Null - Represents an empty or no value.
6. BigInt - Used for very large integers.
7. Symbol - Used for unique identifiers.

Non-Primitive Data Type

1. Object - Used to store collections of data.



OUR LADY OF FATIMA UNIVERSITY - ANTIPOLO
COLLEGE OF COMPUTER STUDIES



LESSON 2

JAVASCRIPT OPERATOR





JavaScript Operators



An operator is simply a symbol that tells JavaScript to perform a specific action on one or more operands (which can be values, variables, or expressions).

1. Arithmetic Operators - Used for mathematical calculations.
2. Assignment Operators - Used to assign values.
3. Comparison Operators - Used to compare values (returns true or false).
4. Logical Operators - Used to combine conditions.
5. Bitwise Operators - Operate on binary numbers.
6. String Operators - Used to combine strings.
7. Conditional (Ternary) Operator - Short form of if-else.



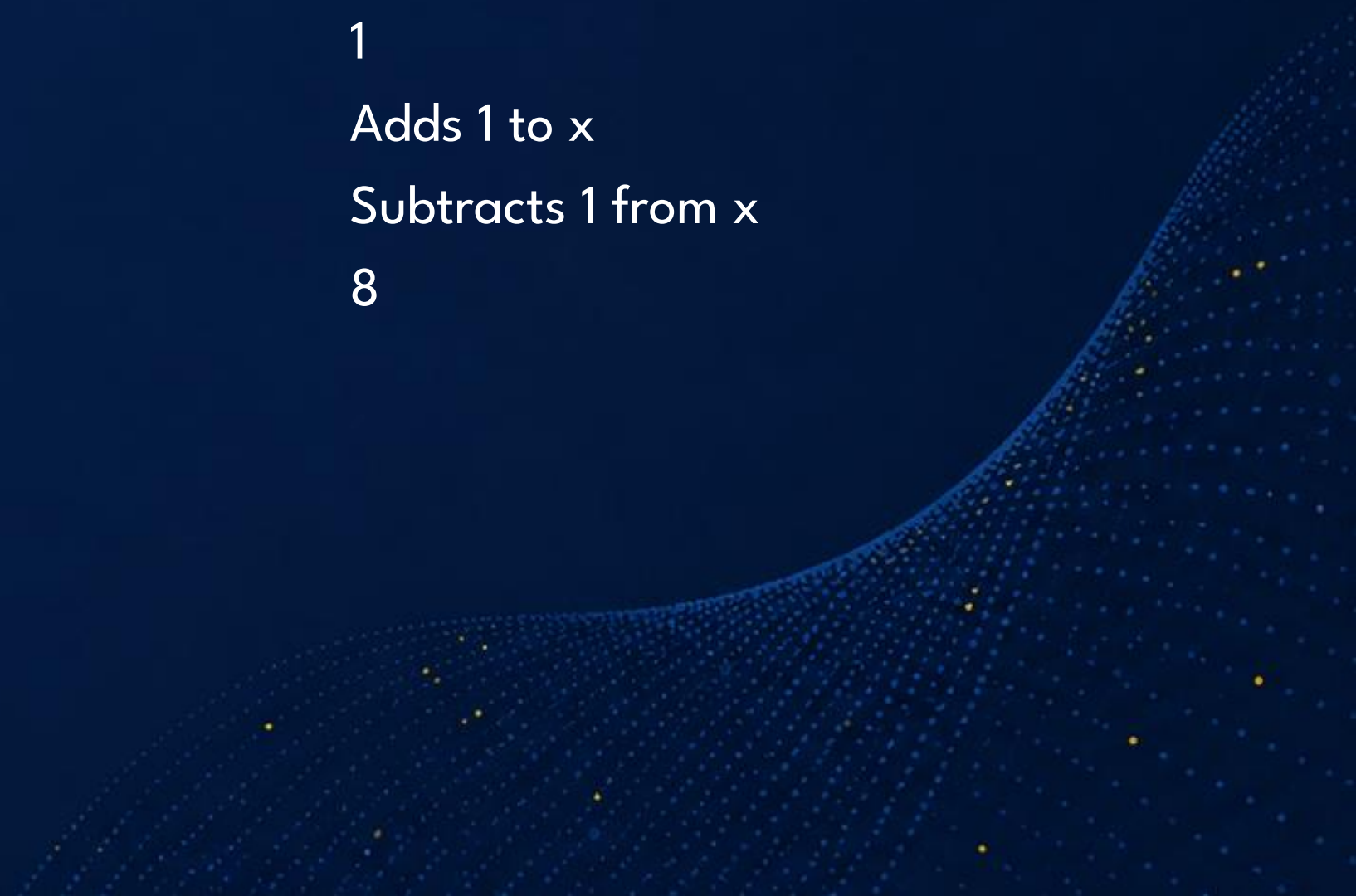


JavaScript Arithmetic Operators



JavaScript Arithmetic Operators are used to perform mathematical calculations on numbers.

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	6 * 2	12
/	Division	15 / 3	5
%	Modulus (Remainder)	10 % 3	1
++	Increment	x++	Adds 1 to x
--	Decrement	x--	Subtracts 1 from x
**	Exponentiation	2 ** 3	8





JavaScript Assignment Operators



JavaScript Assignment Operators are used to assign values to variables.

Operator	Description	Example	Same As
=	Assign value	x = 5	x = 5
+=	Add and assign	x += 3	x = x + 3
-=	Subtract and assign	x -= 2	x = x - 2
*=	Multiply and assign	x *= 4	x = x * 4
/=	Divide and assign	x /= 2	x = x / 2
%=	Modulus and assign	x %= 3	x = x % 3
**=	Exponent and assign	x **= 2	x = x ** 2





JavaScript Comparison Operators



JavaScript Comparison Operators are used to compare two values. They return either true or false.

Operator	Description	Example	Result
==	Equal to (value only)	5 == "5"	true
===	Strict equal to (value and type)	5 === "5"	false
!=	Not equal to	5 != 3	true
!==	Strict not equal to	5 !== "5"	true
>	Greater than	8 > 5	true
<	Less than	4 < 7	true
>=	Greater than or equal to	5 >= 5	true
<=	Less than or equal to	3 <= 6	true





JavaScript Logical Operators



JavaScript Logical Operators are used to combine or evaluate conditions. They return true or false.

Operator	Description	Example	Result
&&	AND (both conditions must be true)	true && false	false
	OR (at least one condition is true)	true false	true
!	NOT (reverses the result)	!true	false





JavaScript Bitwise Operators



JavaScript Bitwise Operators are used to perform operations on binary numbers (bits).

Operator	Description	Example	Result
&	AND	5 & 1	1
	OR	5 1	5
^	XOR	5 ^ 1	4
~	NOT	~5	-6
<<	Left Shift	5 << 1	10
>>	Right Shift	5 >> 1	2
>>>	Unsigned Right Shift	5 >>> 1	2





JavaScript String Operators



JavaScript String Operators are used to combine or manipulate text strings.

Operator	Description	Example	Result
+	Concatenation (joins strings)	"Hello" + " World"	"Hello World"
+=	Append and assign	text += " World"	Adds text to variable





JavaScript Conditional (Ternary) Operators



JavaScript Conditional (Ternary) Operator is a shortcut for an if...else statement. It uses three parts: condition, value if true, and value if false.

`condition ? valueIfTrue : valueIfFalse`

